Projekt 2026 - část 1

předmět Zpracování a vizualizace dat v prostředí Python

Cílem projektu je získat, zpracovat a analyzovat data dostupná na internetu. První část projektu se bude týkat ověření vašeho osvojení technik pro efektivní zpracování, vizualizaci a získání dat.

Orientační struktura projektu

- Část 1 (20 b)
  - efektivní numerické výpočty
  - jednoduchá vizualizace
  - stažení a zpracování dat
- Část 2 (20 b)
  - různé pohledy na data
  - pokročilá vizualizace výsledků
  - zpracování závěrů (porozumění datům)
- Celkový projekt (60 b)
  - znázornění dat na mapě, operace nad těmito daty
  - korelace a predikce
  - automatické vytváření částí zpráv
  - spojení do analytické zprávy

Získání a předzpracování dat (20 bodů)

Vytvořte soubor part01.py, který bude implementovat níže uvedené funkce. Pro zpracování a vizualizaci dat **není povoleno** použít pokročilých knihoven jako je Pandas či Seaborn. Kromě vestavěných knihoven (os, sys, re, gzip, pickle, csv, zipfile…) byste si měli vystačit s: numpy, matplotlib, BeautifulSoup, requests. Další knihovny je možné použít po schválení opravujícím. Pro instalaci virtuálního prostředí venv je možné použít soubor requirements.txt.

Úkol 1: Výpočet částečných součtů Fourierovy řady (5 bodů)

Implementujte funkci `fourier_partial_sums`, která pro jednorozměrné pole časových bodů `t` a počet harmonických složek `n_terms` vypočítá částečné součty Fourierovy řady obdélníkového signálu.

Pro každé `n = 0, …, n_terms - 1` a každý časový bod `t[j]` je výsledek definován jako

$$S[n, j] = \frac{4}{\pi} \sum_{k=0}^{n} \frac{\sin((2k+1)t[j])}{2k+1}$$

Výstupem bude matice `S` o rozměru `(n_terms, len(t))`. Každý řádek obsahuje aproximaci pomocí prvních `n + 1` lichých harmonických složek:

- první řádek obsahuje pouze první harmonickou,
- druhý řádek obsahuje součet první a třetí harmonické,
- poslední řádek obsahuje součet všech `n_terms` lichých harmonických.

Můžete předpokládat, že `t` je neprázdné jednorozměrné pole reálných čísel a `n_terms` je kladné celé číslo.

Pro získání plného hodnocení výpočet vektorizujte pomocí NumPy. Nepoužívejte cykly přes časové body ani harmonické složky. V zadání je dodaná pomalá implementace `fourier_partial_sums_bad`, kterou použijte jako referenci a neupravujte ji. Výsledky obou implementací musí odpovídat s přihlédnutím k přesnosti výpočtů s plovoucí řádovou čárkou.

Doporučený postup: vytvořte pole lichých harmonických a pomocí broadcastingu vypočítejte matici jejich příspěvků ve všech časových bodech. Poté podél vhodné osy vypočítejte kumulativní součet. Běžné funkce NumPy včetně `np.cumsum` jsou povolené; nepoužívejte hotovou knihovní implementaci Fourierovy aproximace.

Prototyp funkce:

```python
def fourier_partial_sums(
    t: NDArray[np.float64], n_terms: int
   ) -> NDArray[np.float64]:
```

Ukázka volání:

```python
t = np.linspace(0, 4 * np.pi, 500)
S = fourier_partial_sums(t, 30)

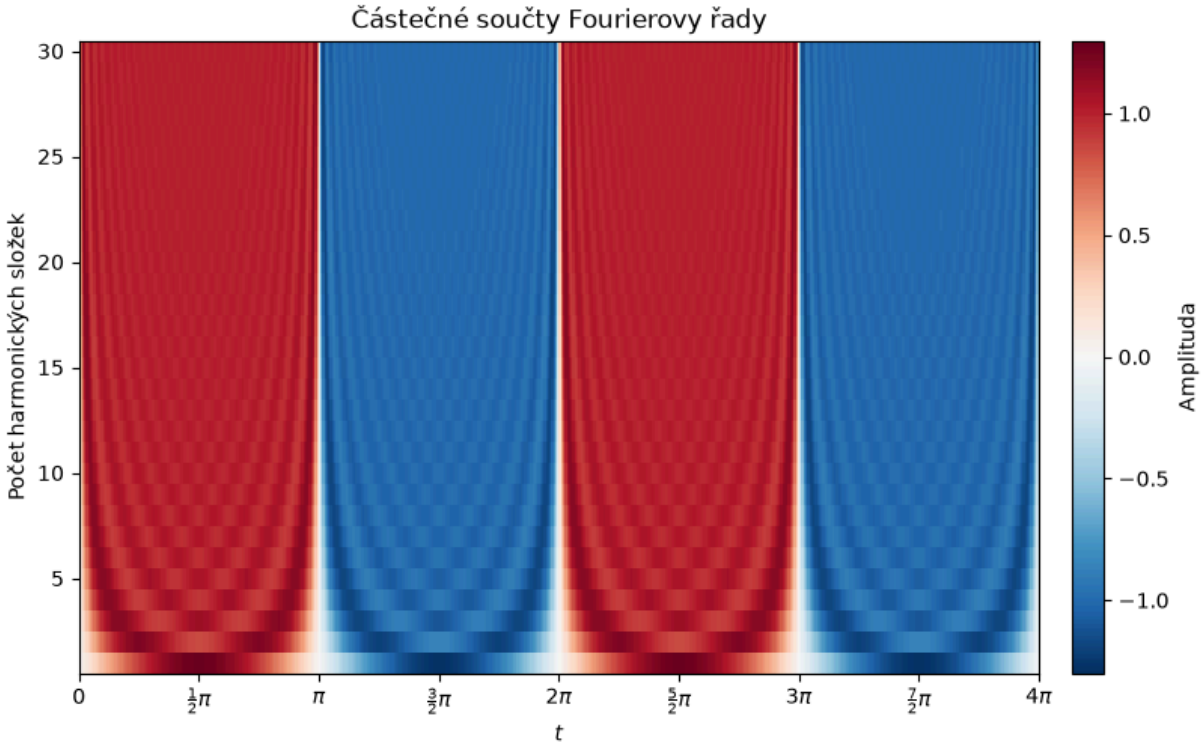
# S.shape == (30, 500)
# S[0] obsahuje první harmonickou.
# S[4] obsahuje součet pěti lichých harmonických.
```

Úkol 2: Vizualizace částečných součtů Fourierovy řady (5 bodů)

Implementujte funkci `plot_fourier`, která zobrazí matici částečných součtů z předchozího úkolu jako barevnou mapu. Pokud nemáte vlastní výpočet implementovaný, můžete použít dodanou referenční funkci `fourier_partial_sums_bad`. Ta bude použita i při testování

Vstupem je matice `S` o rozměru `(n_terms, len(t))` a jednorozměrné pole časových bodů `t`. Každý řádek matice odpovídá počtu použitých harmonických složek a každý sloupec jednomu časovému bodu. Můžete vycházet ze vstupů použitých v zadání - nebudou testovány jiné vstupy.

Graf vykreslete pomocí `imshow` a upravte jej podle vzorového obrázku. Barevnou škálu zvolte sami, obrázek nemusí být přesně stejný, ale je nutné dodržet všechny požadavky stanovené pro kreslení grafů (nepřekrývání, čitelnost atd.).



Argument `show_figure` určuje, zda se má graf zobrazit. Argument `save_path` určuje cestu pro uložení grafu; při hodnotě `None` se graf neukládá. Zobrazení a uložení musí fungovat nezávisle.

Prototyp funkce

```python
def plot_fourier(
    S: NDArray[np.float64],
```

```
    t: NDArray[np.float64],
    show_figure: bool = True,
    save_path: str | None = None,
) -> None:
```

Ukázka použití

```python
t = np.linspace(0, 4 * np.pi, 500)
S = fourier_partial_sums_bad(t, 30)

plot_fourier(
    S,
    t,
    show_figure=True,
    save_path="fourier.png",
)
```

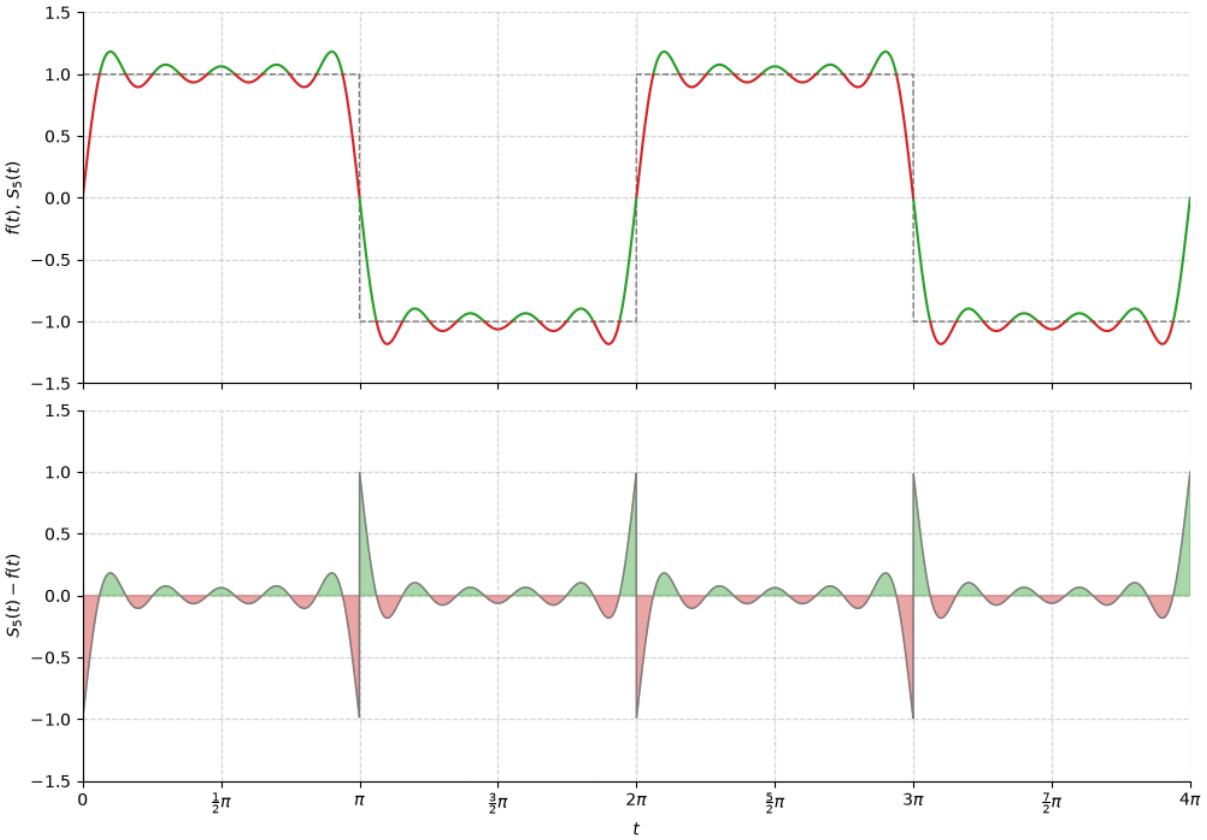
Úkol 3: Vizualizace aproximace obdélníkového signálu (5 bodů)

Vytvořte graf se dvěma podgrafy, který zobrazí aproximaci obdélníkového signálu pomocí Fourierovy řady v rozsahu od 0 do 4π. Můžete vycházet ze vstupů použitých v zadání - nebudou testovány jiné vstupy.

V prvním podgrafu zobrazte ideální signál `y = np.sign(np.sin(t))` čárkovaně a jeho aproximaci pěti lichými harmonickými složkami. Aproximaci `y2` získáte jako poslední řádek matice vrácené funkcí `fourier_partial_sums(t, 5)`. Pokud nemáte vlastní výpočet implementovaný, můžete použít referenční funkci `fourier_partial_sums_bad`.

Aproximaci vykreslete zeleně tam, kde je větší nebo rovna původní funkci, a červeně tam, kde je nižší. Ve druhém podgrafu zobrazte chybu aproximace `y2 - y` a stejnými barvami vyznačte plochu mezi chybou a nulou.

Graf upravte tak, aby vypadal podobně jako na vzorovém obrázku. Použijte sdílené osy x a y, vhodné popisky a značky na časové ose v násobcích π/2 s využitím LaTeXu. Pro argumenty `show_figure` a `save_path` platí stejné podmínky jako ve druhém úkolu.



Prototyp funkce

```python
def generate_approx(
    show_figure: bool = True,
    save_path: str | None = None,
) -> None:
```

Úkol 4: Stažení tabulky (5 bodů)

Ze stránky https://ehw.fit.vutbr.cz/izv/stanice.html stáhněte tabulku meteorologických stanic. Jedná se o kopii stránek ČHMÚ; na oficiální stránky ČHMÚ se nepřipojujte. URL, ze které stahujete, můžete mít uloženou přímo v kódu. Nemusíte zpracovávat přímo uvedenou stránku — pomocí panelu Network v prohlížeči můžete zjistit, odkud se tabulka načítá. Každý sloupec reprezentujte položkou ve slovníku obsahující seznam hodnot ve vhodném datovém typu.

```python
{'positions': ['Cheb',  'Karlovy Vary' ],
 'lats': [50.0683,  50.2016],
```

```
 'longs': [12.3913, 12.9139],
 'heights': [483.0,  603.0]}
```

Výstupem funkce bude slovník se čtyřmi seznamy stejné délky: `positions` obsahuje názvy stanic jako řetězce, `lats` zeměpisné šířky, `longs` zeměpisné délky a `heights` nadmořské výšky v metrech. Číselné hodnoty vraťte jako `float`. Položky na stejném indexu musí odpovídat stejné stanici; zachovejte pořadí stanic ze zdrojové tabulky.

Výstup základním způsobem ověřuje přiložený soubor `test_part01.py`. Můžete předpokládat, že se struktura zdrojové stránky při testování nezmění. Data načítejte pouze ze stránek https://ehw.fit.vutbr.cz/izv.

Prototyp funkce

```python
def download_data() -> Dict[str, List[Any]]:
```

Využití AI nástrojů

Zakazujeme používat generativní jazykové modely (LLM) k vytváření řešení úloh — cílem projektu je, abyste si postupy sami osvojili. Vhodným použitím AI je například revize vlastního kódu, hledání dokumentace nebo vysvětlení funkcí a chybových hlášek.

V řetězci `AI_USAGE` v souboru `part01.py` stručně popište použité nástroje, účel jejich použití a části řešení, kterých se týkalo. Pokud jste AI nepoužili, výslovně to uveďte.

Za správnost odevzdaného řešení odpovídáte vy a musíte mu rozumět. Kteréhokoli studenta mohu požádat o vysvětlení funkčnosti jeho kódu a zvoleného postupu.

Testování

K ověření funkčnosti můžete použít vlastní testovací sekvenci v bloku `if __name__ == "__main__":` nebo přiložené testy v souboru `test_part01.py`. Testy spustíte z adresáře s oběma soubory příkazem:

```shell
python3 -m pytest
```

Testy kontrolují několik hodnot výpočtu, vytvoření souborů s grafy a základní správnost stažených dat. Úspěšné absolvování testů je podmínkou nutnou, nikoli dostačující pro plné hodnocení. Vizuální podobu grafů, efektivitu a kvalitu kódu posuzujeme samostatně.

Dokumentace všech částí (souborů, funkcí) bude přímo v odevzdaném souboru. Snažte se dodržovat konvenci PEP 257 [https://www.python.org/dev/peps/pep-0257] a PEP 8 [https://www.python.org/dev/peps/pep-0008/]. Pozor, Python má přímo definovaný kódovací styl (narozdíl od C), a proto kontrola bude součástí hodnocení (1 bod).

Odevzdávání a hodnocení

Do **1. 11. 2026** odevzdejte jeden soubor `part01.py`.

Hodnotit se bude zejména:

- správnost výsledků
- vizuální dojem z grafů
- kvalita kódu
    - efektivitu implementace a reprezentace (i rychlost v porovnání s ostatními řešeními), využívání efektivních funkcí (např. NumPy)
    - přehlednost kódu
    - dodržení standardů a zvyklostí pro práci s jazykem Python - dokumentační řetězce, splnění PEP8
    - dokumentace funkcí a souborů
    - znovupoužitelnost kódu — potřebné části oddělte do funkcí; při importu souboru se nesmí spouštět ukázkové výpočty, vytvářet grafy ani stahovat data.

Celkem za první část můžete získat až 20 bodů, přičemž je k zápočtu nutné získat z této části minimálně 1 bod.

Dotazy a připomínky

Na mailu mrazek@fit.vutbr.cz či osobně po přednáškách. Při pozdním odevzdání (na email) bude počítána penalizace -5 bodů za každý započatý den.